

SISTEMAS OPERACIONAIS – 2025

GUIA RÁPIDO DE GERENCIAMENTO DE ARQUIVOS NO LINUX

Prof. Ana Paula Costacurta

INTRODUÇÃO

Gerenciar arquivos e pastas no Linux usando a linha de comando é importante. Um arquivo é um conjunto de dados com um nome e algumas características, como a data em que foi modificado. Por exemplo, quando você transfere fotos do seu telefone para o computador e dá a elas nomes significativos, cria arquivos de imagem. Diretórios são como pastas para organizar arquivos. Pense neles como gavetas dentro de gavetas, onde você pode colocar mais gavetas.

Usar a linha de comando no Linux é a maneira mais eficiente de lidar com arquivos. As ferramentas disponíveis tornam as tarefas mais rápidas do que usar um programa visual. Alguns comandos importantes são: `ls` (listar), `mv` (mover), `cp` (copiar), `pwd` (mostrar diretório atual), `find` (procurar), `touch` (criar ou atualizar), `rm` (remover), `rmdir` (remover diretório vazio), `echo` (mostrar texto) e `mkdir` (criar diretório). Esses comandos ajudam a organizar e gerenciar arquivos e pastas de forma simples e direta.

CRIANDO DIRETÓRIOS

Para criar diretórios, usamos o comando **`mkdir`**. Vamos ver como criar um novo diretório dentro do nosso diretório pessoal:

Primeiro, vamos para o nosso diretório pessoal usando o comando abaixo:

```
$ cd ~
```

Agora, vamos supor que estamos `/home/codespace`, para verificar em qual diretório estamos utilizamos o comando abaixo:

```
$ pwd
```

```
/home/codespace
```

Para listarmos o conteúdo desse diretório, suponha que temos os diretórios `java` e `nvm`, usamos o comando abaixo:

```
$ ls
```

```
java nvm
```

IMPORTANTE:

O sistema de arquivos do Linux diferencia maiúsculas de minúsculas nos nomes de arquivos e diretórios, ao contrário do Microsoft Windows. Isso significa que nomes como /etc/ e /ETC/ se referem a diretórios distintos. Experimente os comandos a seguir:

```
$ cd /
$ ls
bin boot dev etc go home lib lib32 lib64 libx32 media mnt opt proc
root run sbin srv sys tmp usr var vscode workspaces
$ cd ETC
bash: cd: ETC: No such file or directory
$ pwd
/
$ cd etc
$ pwd
/etc
```

DICA:

Ao listar o conteúdo de um diretório com o comando `ls`, as opções `-la` são usadas para mostrar todos os arquivos e diretórios, incluindo os ocultos, e exibir detalhes sobre eles em um formato longo. Aqui está o significado de cada opção:

- `-l` indica que queremos uma saída em formato longo, o que significa que serão exibidos detalhes sobre cada arquivo ou diretório, como permissões, número de links, proprietário, grupo, tamanho e data de modificação.
- `-a` indica que queremos mostrar todos os arquivos, incluindo os ocultos, que começam com um ponto (.) no nome.

Na sequência, vamos abrir o github para executar os comandos para realizar a visualização da execução.

ATIVIDADE 1

Instruções para execução:

- 1) Abrir o repositório do projeto Suporte TI no GitHub, utilizado na última atividade.
- 2) Abrir o Codespace no repositório.
- 3) Executar os comandos abaixo, na sequência abaixo apresentada, no prompt do terminal do codespace do github do projeto Suporte ti que acabou de abrir.

```
$ cd ~  
$ cd /  
$ pwd  
/home/codespace  
$ ls  
java nvm  
$ ls -la
```

Evidências: Tirar print da tela com os resultados dos comandos executados na atividade 1. Coloque o arquivo como: print-atividade1.png

Na sequência, vamos para o nosso diretório /workspaces/suporte-ti, utilizando o comando abaixo:

ATIVIDADE 2

Instruções para execução:

- 1) Listar a pasta do seu workspace

```
$ cd /  
$ ls  
$ cd workspace  
$ ls  
suporte-ti
```

Observação: o nome do diretório “suporte-ti” que irá aparecer depende do nome do diretório que você criou como nome do seu repositório.

Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade2.png

Na sequência, vamos criar um diretório chamado “Aula_SO_2025” dentro da pasta /workspaces/suporte-ti, utilizando o comando abaixo:

ATIVIDADE 3

Instruções para execução:

- 1) Criar um diretório na pasta /workspace/suporte-ti

```
$ cd /suporte-ti
$ mkdir Aula_SO_2025
$ ls -la
$ cd /Aula_SO_2025
```

Observação: o nome do diretório “suporte-ti” que irá aparecer depende do nome do diretório que você criou como nome do seu repositório.

Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade3.png

Agora que dominamos a criação de diretórios, é hora de dar o próximo passo e aprender a criar arquivos.

CRIANDO ARQUIVOS

Vamos falar sobre como criar arquivos. Geralmente, os programas criam arquivos para armazenar dados. Podemos criar um arquivo vazio usando o comando **touch**. Se usarmos o **touch** em um arquivo que já existe, ele não mudará o conteúdo, apenas atualizará o horário da última modificação.

Podemos criar alguns arquivos usando o comando touch:

```
$ touch exercicio1
$ ls
exercicio1
```

Como o **touch** cria arquivos vazios, não veremos nada, quando executar o comando **cat**:

```
$ cat exercicio1
```

Assim podemos usar o “**echo** >” para criar um arquivo de texto simples:

```
$ echo Hello World > exercicio-1.txt
$ cat exercicio1
Hello World
```

O comando **echo** mostra texto na linha de comando. O símbolo > instrui o sistema a escrever a saída de um comando no arquivo especificado, em vez de mostrá-la no terminal. Isso significa que o texto "Hello World" é escrito no arquivo exercicio1. É

importante ter cuidado com o >, pois se já houver um arquivo com o mesmo nome, ele será sobrescrito.

ATIVIDADE 4

Instruções para execução:

- 1) Criar um arquivo em branco e incluir um texto no arquivo.

```
$ touch exercicio-1.txt
$ ls
exercicio1-1.txt
$ cat exercicio-1.txt
$ echo Hello World > exercicio-1.txt
$ cat exercicio-1.txt
Hello World
$
```

Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade4.png

RENOMEANDO ARQUIVOS

O comando **mv** serve para mover e renomear arquivos. Para isso crie alguns arquivos para praticar. Neste momento, você já deve estar familiarizado com estes comandos:

ATIVIDADE 5

Instruções para execução:

- 1) Criar uma pasta exercicio2 dentro da Aula_SO_2025

```
$ mkdir exercicio2
$ ls
$ ls -la
```

Observação: o nome do diretório “suporte-ti” que irá aparecer depende do nome do diretório que você criou como nome do seu repositório.

Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade5.png

```
$ cd exercicio2
$ touch exer11 exer22
```

```
$ mv exer11 exercicio11
$ mv exer22 exercicio11
$ echo texto3 > texto3
```

Vamos usar o comando **mv nome_arquivo_atual novo_nome_arquivo** para renomear um arquivo, a sintaxe do comando tem como o primeiro argumento o primeiro argumento é o nome atual (**nome_arquivo_atual**), o segundo é o novo nome (**novo_nome_arquivo**):

ATIVIDADE 6

Instruções para execução:

- 1) Criar dois arquivos para após renomealos.

```
$ cd exercicio2
$ touch exerc11 exerc22
$ ls
$ mv exer11 exercicio11
$ ls
exerc22 exercicio11
$ mv exerc22 exercicio11
$ ls
exercicio11
```

Observação: Quando utilizamos o comando “**mv exerc22 exercicio11**” movendo um arquivo para o arquivo exercicio11 que já existia o mesmo foi sobrescrito sem nenhum aviso.

Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade6.png

Tenha cuidado ao usar o comando mv. Se o novo nome já estiver atribuído a um arquivo existente, ele será sobrescrito. Use a opção -i para que mv avise caso você esteja prestes a sobrescrever um arquivo existente. Utilize y se desejar substituir.

ATIVIDADE 7

Instruções para execução:

- 1) Criar o arquivo exerc22 para renomear com o comando mv utilizando o parâmetro -i.

```
$ clear
$ ls
$ touch exerc22
$ ls
$ mv -i exer22 exercicio11
mv: overwrite `exercicio11'? n
```

	<pre>\$ ls exer22 exercicio11 \$ mv -i exer22 exercicio12 \$ ls exercicio11 exercicio12</pre>	
Observação: Responda não (n) a pergunta de sobrescrever (overwrite) o arquivo. Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade7.png		

MOVENDO ARQUIVOS

Os arquivos são movidos de um diretório para outro com o comando mv.

ATIVIDADE 8		
Instruções para execução:		
1) Criar diretório e mover arquivos para dentro dos diretórios		
	<pre>\$ clear \$ mkdir d1 d2 \$ ls d1 d2 exercicio11 exercicio12 \$ mv exercicio 11 d1 \$ ls d1 d2 exercicio12 \$ cd d1 \$ ls exercicio11 \$ cd .. \$ ls d1 d2 exercicio12 \$ mv exercicio12 d2 \$ ls d1 d2 \$ cd d2 \$ ls exercicio12</pre>	
Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade8.png		

ATIVIDADE 9		
Instruções para execução:		
1) Mover diretórios para dentro de outro diretório.		

	<pre>\$ clear \$ cd .. \$ ls d1 \$ cd d1 \$ ls d2 exercicio11 \$ cd d2 \$ ls exercicio12</pre>	
Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade9.png		

EXCLUINDO ARQUIVOS E DIRETÓRIOS

- ‘ O comando **rm** serve para excluir arquivos.

ATIVIDADE 10 Instruções para execução: 1) Excluir arquivos		
	<pre>\$ clear \$ cd ../../.. \$ mkdir exercicio3 dir3 dir2 \$ ls dir2 dir3 exercicio-1.txt exercicio2 exercicio3 \$ cd exercicio3 \$ touch exer3 exercicio33 \$ ls exer3 exercicio33 \$ rm exer3 \$ ls exercicio33 \$ rm -i exercicio33 rm: remove regular empty file `exercicio22'? n \$ ls exercicio3</pre>	
Observação: Responda não (n) a pergunta de remover (remove) o arquivo. Quando utilizamos o comando rm sem o parâmetro -i ele não irá fazer a pergunta de confirmação se desejamos excluir o arquivo. Evidências: Tirar print da tela com os resultados dos comandos. Coloque o arquivo como: print-atividade10.png		

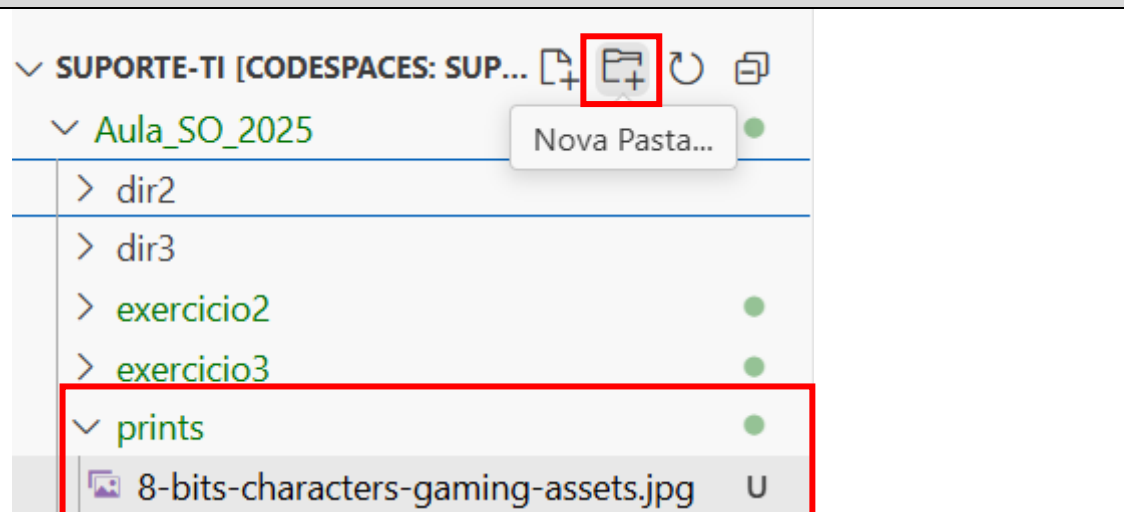
DICA

o comando rm tem uma opção -i para exibir um aviso antes de executar qualquer ação.

ATIVIDADE 12

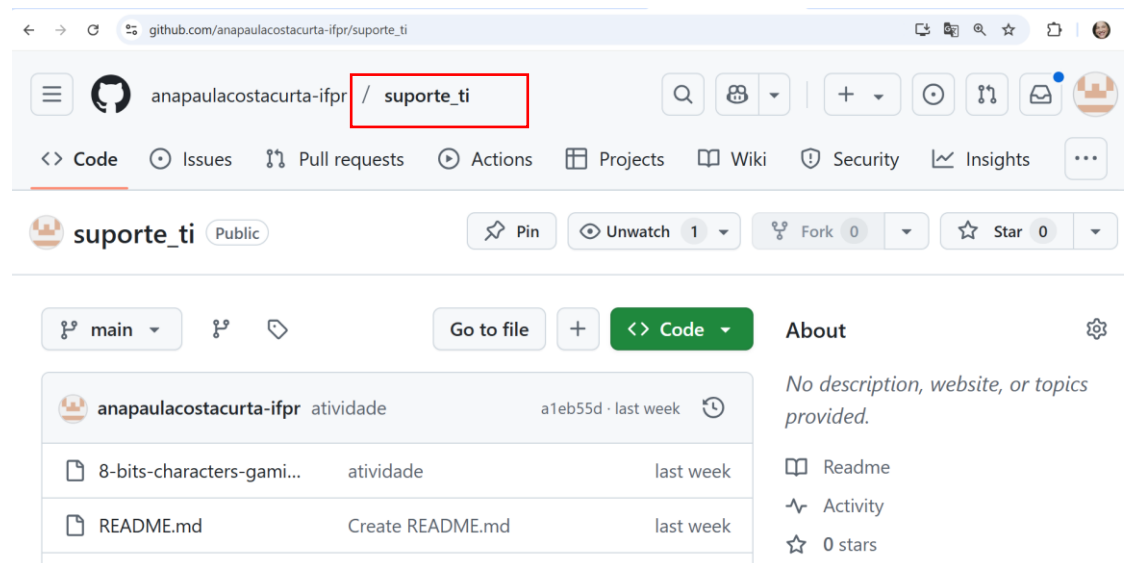
Instruções para execução:

- 1) Criar uma pasta chamada prints dentro de Aula_SO_2025



- 2) Incluir os 10 prints das atividades dentro da pasta prints criadas.

Observação: A entrega da Atividade deve ser apenas o link do github do seu repositório. Para isso você deve clicar no nome do seu repositório.



E após isso você deve copiar o endereço que vai aparecer no navegador.

github.com/anapaulacostacurta-ifpr/suporte_ti

anapaulacostacurta-ifpr / suporte_ti

<> Code Issues Pull requests Actions Projects Wiki Security Insights

suporte_ti Public Pin Unwatch 1 Fork 0 Star 0

main Go to file + <> Code About

anapaulacostacurta-ifpr atividade a1eb55d · last week

8-bits-characters-gami...	atividade	last week
README.md	Create README.md	last week

No description, website, or topics provided.

Readme Activity 0 stars

Esse endereço você deverá incluir como link na resposta da atividade.